

Genetic Algorithm and Machine Learning Timetable Optimization System

DJSCE Data Science Department

Semester IV | Divisions D1, D2, D3

Hybrid Architecture: Genetic Algorithm + Machine Learning + Database + Frontend

Overall System Flow:

```
graph TD; A[Data Ingestion] --> B[ML Model Preparation]; B --> C[GA Initialization]; C --> D[Evolution Loop]; D --> E[Best Timetable]; E --> F[Store + Visualize];
```

Block 1: Data Ingestion Layer

Admin inputs subjects, credits, weekly hours, subject types, faculty mapping, rooms, time slots and constraints.

Backend loads divisions, subject metadata, faculty availability, room capacity and threshold parameters.

Preprocessing builds structured in-memory models.

```
// Division Time Grid
schedule[division][day][slot]

// Faculty Availability
facultyAvailability[facultyId][day][slot]

// Room Availability
roomAvailability[roomId][day][slot]

// Weekly Requirement Tracker
requiredHours[division][subjectId]
```

Output: structured context object containing all scheduling metadata.

Block 2: Machine Learning Model Preparation

Three predictive models are loaded before GA execution.

Difficulty Model predicts academic load intensity per subject or day.

Fatigue Model predicts cognitive strain based on subject sequence and daily structure.

Infrastructure Stress Model predicts laboratory overload and operational congestion.

```
difficulty_score = difficultyModel.predict(features)
fatigue_index = fatigueModel.predict(dayFeatures)
infra_stress_score = infraModel.predict(weekFeatures)
```

All outputs are normalized between 0 and 1 and used during fitness evaluation.

Block 3: Initial Population Generation

Population size example: 80 chromosomes. Each chromosome represents complete weekly schedule for D1, D2 and D3.

Labs are allocated first due to consecutive slot and batch constraints.

Theory subjects are placed respecting weekly credit requirements.

Repair functions fix hard violations before fitness scoring.

```
assignBreakSlots(schedule)
placeConsecutiveLabBlocks()
assignTheorySubjects()
repairTeacherClashes()
repairRoomClashes()
```

Block 4: GA Evolution Loop

For each generation, selection, crossover, mutation, repair and evaluation are performed.

```
function tournamentSelection(population) {
  const sample = pickRandom(3)
  return lowestFitness(sample)
}

if (Math.random() < crossoverRate) {
  child = divisionWiseCrossover(parentA, parentB)
}

if (Math.random() < mutationRate) {
  mutate(child)
}
```

Fitness integrates constraint penalties and ML-based quality penalties.

```
fitness =
  10000 * hardPenalty
+ 100 * softPenalty
+ 40 * difficultyPenalty
+ 60 * fatiguePenalty
+ 30 * infraPenalty
```

Block 5: Final Timetable Selection

```
bestChromosome = population.reduce(minFitness)
```

Selected timetable has zero hard violations, balanced difficulty, low fatigue and distributed lab stress.

Block 6: Persistence and Output

```
const output = serialize(bestChromosome)
await Timetable.create(output)
```

Frontend generates division view, faculty view and room view. Optional PDF export and analytics dashboard available.

System Layer Interaction Summary

Database: Stores structured metadata.

ML Models: Predict dynamic academic and infrastructure quality.

GA Engine: Optimizes timetable structure.

Fitness Layer: Integrates constraints and predictive intelligence.

Adaptive Module: Controls convergence.

Frontend: Visualization and administration.